



第三章 同态加密

目 录

『04』

知识点四：类同态BGN方案

『05』

知识点五：全同态典型方案

『06』

知识点六：开发框架SEAL

1. 群

- 群是一种由**元素的集合**和一个**二元运算**组成的基本代数结构。
- 若元素集合 G 和二元运算“ $\cdot$ ”满足封闭性、结合律、单位元和逆元素四个要素，则称之为群。
 - **封闭性**：对于所有集合 G 中的元素 a 和 b ， $a \cdot b$ 的结果也在集合 G 中；
 - **结合律**： $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ 对任意 $a, b, c \in G$ 成立；
 - **单位元**：集合 G 存在元素 e ，满足 $e \cdot a = a \cdot e = a, \forall a \in G$ ；
 - **逆元素**：对于集合 G 中的任意一个元素 a ，存在集合 G 中的另一个元素 b 使 $a \cdot b = b \cdot a = e$ 。
- 定义一个有限群的阶为群中元素的个数。
- 用 $\cdot$ 来代指这个二元运算，并定义元素的乘方 a^2 为 $a \cdot a$ ，并以此推演出元素的更高次方。

1. 群

- **交换群**：若一个群还满足交换律（即 $a \cdot b = b \cdot a$ ，任意 $a, b \in G$ 成立），则可进一步称其为交换群或阿贝尔群。
- **循环群**：若一个群 G 的每一个元素都可以被表达成群 G 中某一个元素 g 的次方 g^m ，则称 G 为循环群，记作 $G = \langle g \rangle = \{g^m \mid m \in \mathbb{Z}\}$ ， g 被称为 G 的一个生成元，因为可以通过对 g 的不断自我运算来获得群中的所有元素。
注意：符号 $\langle g \rangle$ 与符号 (g) 相同，也经常被定义为由 g 生成的循环群。
- 在一个有限群中，如果对不是生成元的其他元素 a 进行这种次方运算，它最终会循环遍历一个群 G 的**子集**。
- 可以证明，所有元素的这种遍历都会经过单位元 e 。将满足 $a^n = e$ 的最小正整数 n 称为 a 元素的阶，生成元的阶和群的阶相等。

1. 群

- 以整数模6加法群 $Z_6=\{0, 1, 2, 3, 4, 5\}$ 为例, 其群的阶为6, 单位元为0。6个元素的阶分别是1、6、3、2、3、6。其中1、5为生成元。
- **以元素5为例, 经过模加运算有:**
$$5^1 = 5,$$
$$5^2 = (5+5) \pmod{6} = 4,$$
$$5^3 = (5+5+5) \pmod{6} = 3,$$
$$5^4 = (5+5+5+5) \pmod{6} = 2,$$
$$5^5 = (5+5+5+5+5) \pmod{6} = 1,$$
$$5^6 = (5+5+5+5+5+5) \pmod{6} = 0 = e.$$
- 故称元素5的阶为6, 为群G的生成元。

1. 群

- 很容易可以发现循环群的一个特性，即

“由生成元 g 构建的循环群很容易满足加法同态”：

$$g^{r_1} \cdot g^{r_2} = g^{(r_1+r_2)}。$$

- 根据上述例子，也很容易就可以验证：

对于明文1和2，对应的密文是 5^5 和 5^4 ，因为这里的运算符“ $\cdot$ ”就是加法，显然，

$$5^5 \cdot 5^4 = (5 \cdot 9 = 5 \cdot 6 + 5 \cdot 3) \pmod{6} = 5^3 = 5^{(1+2)} = 3。$$

整数模5加法群 $Z_5 = \{0, 1, 2, 3, 4\}$ ，其群的阶为5，单位元为0。
元素2的阶为

A 2

B 3

C 4

D 5

2. 环

- 在群的基础上，还可以使用两种运算和元素集合 R 来构建环，这两种运算一般写作 “+” 和 “·”。
- 通常可以这么理解这两个运算：
- “+” 一般表示环上的加法，其对应的单位元通常为0，由其定义的群为加法群，群上的两个相同运算的+运算，可以记作 $a + a = 2a$;
- “·” 一般表示环上的乘法，其对应的单位元通常记作 e ，由其定义的群为乘法群，群上的两个相同运算的·运算，可以记作 $a \cdot a = a^2$ 。

2. 环

- 环可以看作建构在**交换群之上另外增加了一个运算“ $\cdot$ ”**，除了原有运算符号+的群的性质和交换律，在新的运算符号上还需要满足：
- **封闭性**：对于所有 R 中元素 a 和 b ， $a \cdot b$ 的结果也在 R 中；
 - **结合律**： $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ 对任意 $a, b, c \in R$ 成立；
 - **单位元**： R 存在元素 e ，满足 $e \cdot a = a \cdot e = e$ ， $\forall a \in R$ ，该元素常被称作乘法单位元；
 - **分配律**：乘法操作可以在加法之间进行分配。即给出任意 $a, b, c \in R$ ，有 $a \cdot (b + c) = a \cdot b + a \cdot c$ 和 $(b + c) \cdot a = b \cdot a + c \cdot a$ 。
 - 事实上，满足上述封闭性和结合律的 $(R, \cdot)$ 构成一个半群。再加上单位元，就构成一个么半群。

2. 环

- $(R, +, \cdot)$ 构成一个环 (Ring) , 当:
 - $(R, +)$ 构成交换群;
 - $(R, \cdot)$ 构成么半群;
 - $(R, +, \cdot)$ 满足分配律。
- 在一个环 $(R, +, \cdot)$ 中, 若其子集 I 与其加法构成子群 $(I, +)$, 且满足 $\forall i \in I, r \in R, i \cdot r \in I$, 则称 I 为环 R 的一个**右理想** (Right Ideal) 。若 $\forall i \in I, r \in R, r \cdot i \in I$ 则称 I 为环 R 的一个**左理想** (Left Ideal) 。若同时满足左右理想, 则称 I 为环 R 上的一个**理想** (Ideal) 。
- **理想对内具有乘法封闭性, 对外具有乘法吸收性。**

3. 域

域是环的一种特殊形式，它要求乘法运算必须满足交换律，因此域是交换环，还要求，域中的除0外的所有元素都有乘法逆元素。

有限域，顾名思义，就是一个包含有限个元素的集合。一个有限域的具体例子就是模 p 的整数域，其中 p 是一个素数，通常它可表示为 Z_p 等。

从群到环，再到域，是一个条件逐渐收敛的过程。

4. 双线性映射

- 一个双线性映射是由两个向量空间上的元素，生成第三个向量空间上一个元素的函数，并且该函数对每个参数都是线性的。
- 理解：若 $B: V \times W \rightarrow X$ 是一个双线性映射，则 V 固定， W 可变时， W 到 X 的映射是线性的； W 固定， V 可变时， V 到 X 的映射也是线性的；也就是说，保持双线性映射中的任意一个参数固定，另一个参数对 X 的映射都是线性的。

4. 双线性映射

存在一个**加法循环群** G_1 和**乘法循环群** G_2 ，这两个群的阶都为大素数 q 。定义 $e: G_1 \times G_1 \rightarrow G_2$ 为这两个循环点群之间的一个双线性映射，且该映射满足如下三个性质：

(1) 双线性：对于所有的 $P, Q \in G_1$ 和 $a, b \in Z_q^*$ ，有

$$e(aP, bQ) = e(P, Q)^{ab}$$

$$e((a + b)P, Q) = e(P, Q)^a \cdot e(P, Q)^b$$

注，这里的 Z_q^* 表示不包含0的整数集， Z 表示整数集， q 表示阶， $*$ 表示不包含0元素。

(2) 非退化性： e 为非平凡映射，即 e 不会将 $G_1 \times G_1$ 的所有值映射到 G_2 的单位元。

(3) 可计算性：具有有效的算法对于任何的 $P, Q \in G_1$ 能够计算 $e(P, Q)$ 。

满足如上三个性质的双线性映射就叫做可采纳的双线性映射。

Boneh等给出了关于双线性映射更具体的描述，提出了与双线性相关的数学难题，并用于设计基于身份加密、基于属性的加密等密码原语。

5. 子群判定问题

子群判定问题是指：

给定 (n, G, G_1, e) ，其中群 G, G_1 具有相同的阶 $n = pq$ ，

$e: G \times G \rightarrow G_1$ 是一个双线性映射，

给定一个元素 $x \in G$ ，如果 x 的阶是 p ，则输出 1，否则输出 0。

上述问题也可以描述为：

一个阶为 $n = p * q$ (p, q 为质数) 的合数阶群里，

判定一个元素是否属于某个阶为 p 的子群的问题。

该判定问题为困难问题，BGN方案的实现就是基于子群决策问题。

6. 方案构造

BGN能够同时支持加法和乘法的关键原因在于：

- ① 它提出了一套能够构建在两个群 G 和 G_1 之间的双线性映射 $e: G \times G \rightarrow G_1$ 的方法。

BGN提出的方法能够生成两个阶相等的乘法循环群 G 和 G_1 ，并建立其双线性映射关系 e ，且满足当 g 是 G 的生成元时， $e(g, g)$ 为 G_1 的生成元。

- ② 在执行乘法之前，密文属于群 G 中的元素，可以利用群的二元操作进行密文的加法同态操作。

- ③ 密文的乘法同态操作通过该双线性映射函数，将密文从群 G 映射到 G_1 的元素当中。执行乘法同态操作之后，处于 G_1 的密文仍然能够继续使用同态加法。

6. 方案构造

(1) 密钥生成

给出安全参数，选择大质数 q_1, q_2 并获得合数 $n = q_1 q_2$ ，BGN将构建两个阶为 n 的循环群 G, G_1 和双线性映射关系 $e: G \times G \rightarrow G_1$ 。

从 G 中随机选取两个生成元 g, u ，并获得 $h = u^{q_2}$ 。可知 h 为某阶为 q_1 的 G 的子群的生成元。

公钥设置为 (n, G, G_1, e, g, h) ，私钥设置为 q_1 。

$$(u^{q_2})^{q_1} = u^n = e$$

(2) 加密

对于消息明文 m （某小于 q_2 的自然数），随机抽取0到 n 之间的一个整数 r ，生成密文：

$$c = E(m) = g^m h^r \in G$$

(3) 解密

使用私钥 q_1 ，首先计算：

$$c^{q_1} = (g^m h^r)^{q_1} = (g^m u^{q_2 r})^{q_1} = g^{mq_1} u^{q_2 q_1 r} = g^{mq_1} u^{nr} = (g^{q_1})^m,$$

然后，计算离散对数得到明文： $m = \log_{g^{q_1}}(c^{q_1})$ 。

6. 方案构造

(1) 密文上的同态加法性质

由生成元 g 构建的循环群很容易构造加法同态, $g^{r_1} \cdot g^{r_2} = g^{(r_1+r_2)}$ 。

对于两个密文 $c_1 = g^{m_1} \cdot h^{r_1}$ 和 $c_2 = g^{m_2} \cdot h^{r_2}$, 很明显 $c_1 \cdot c_2 = g^{(m_1+m_2)} \cdot h^{(r_1+r_2)}$ 。

(2) 密文上的同态乘法

密文上的同态乘法, 则通过双线性映射函数实现, $e(u^a, v^b) = e(u, v)^{ab}$ 。

在密钥生成的时候, 定义 $g_1 = e(g, g)$ 和 $h_1 = e(g, h)$, 且将 h 写作 $h = g^{\alpha q_2}$ (因为 g 可生成 $u: u = g^a$), 定义对 c_1 和 c_2 的同态乘法运算为:

$$e(c_1, c_2)h_1^r = e(g^{m_1}h^{r_1}, g^{m_2}h^{r_2})h_1^r = g_1^{m_1m_2}h_1^{\hat{r}} \in G_1。$$

式中, $\hat{r}$ 是前文提到的随机抽取的0到 n 之间的整数 r 。

由此可见, 经过同态乘法之后的密文从 G 转移到了 G_1 , 其解密过程在 G_1 上完成, 即 G_1 的生成元 $g_1 = e(g, g)$ 替代

推导过程省略

显然计算是乘法同态

有关方案BGN，说法错误的是

- ☐ A 属于类同态加密
- ☐ B 支持任意多次乘法，但仅支持一次加法
- ☐ C 所采用的双线性映射为可采纳的双线性映射
- ☐ D 循环群比较容易构造加法同态

目 录

『04』

知识点四：类同态BGN方案

『05』

知识点五：全同态典型方案

『06』

知识点六：开发框架SEAL

1. 格

(1) 格的定义

给定一个 n 维向量空间 R^n ，格（Lattice）是其上的一个离散加法子群。

根据线性代数知识，可以构造一组 n 个线性无关的向量 $v_1, v_2, v_3, \dots, v_n \in R^n$ 。基于该组向量的整数倍的线性组合，可以生成一系列的离散点：

$$L(v_1, v_2, v_3, \dots, v_n) = \left\{ \sum_{i=1}^n \alpha_i v_i \mid \alpha_i \in \mathbb{Z} \right\}$$

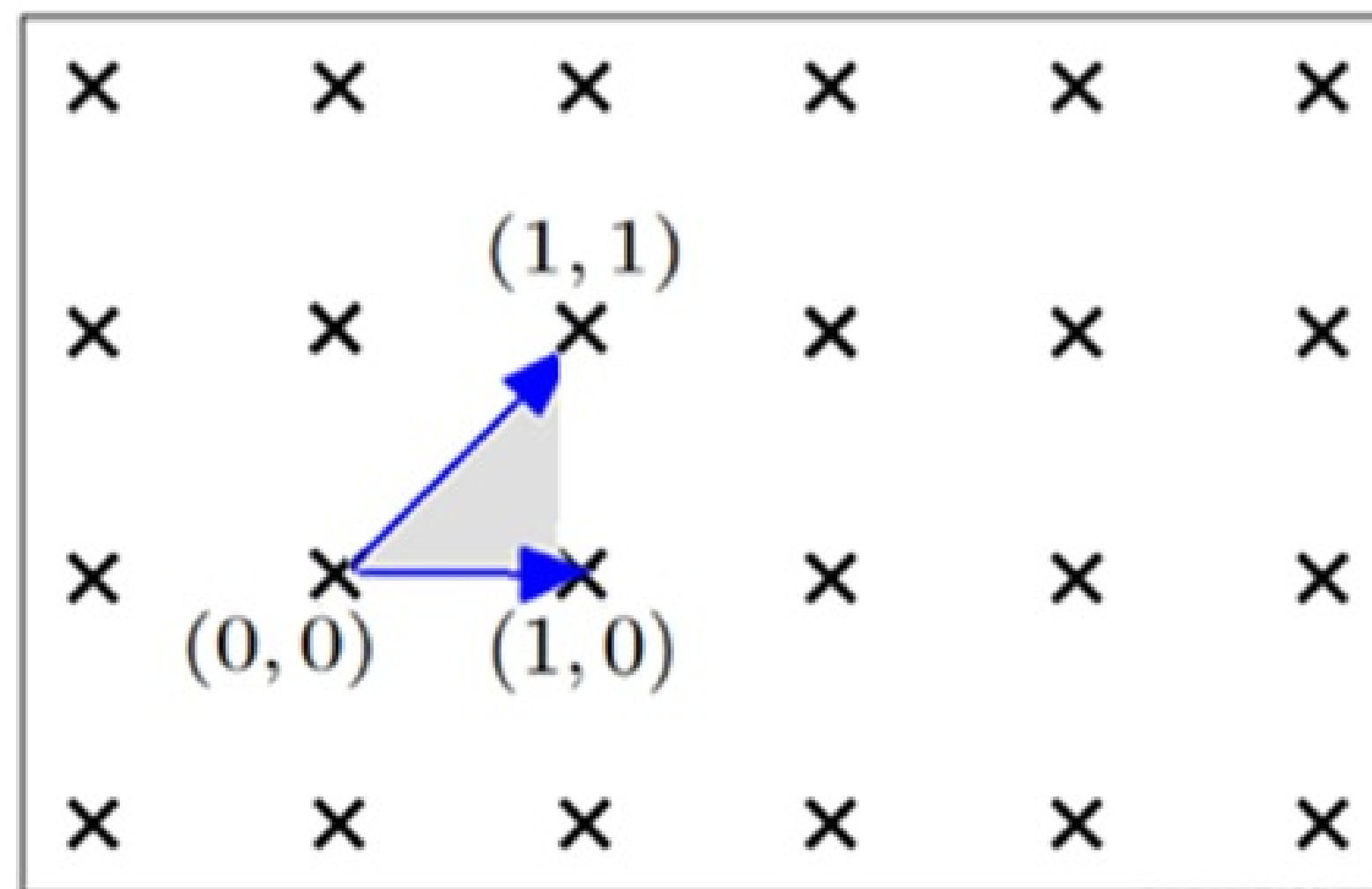
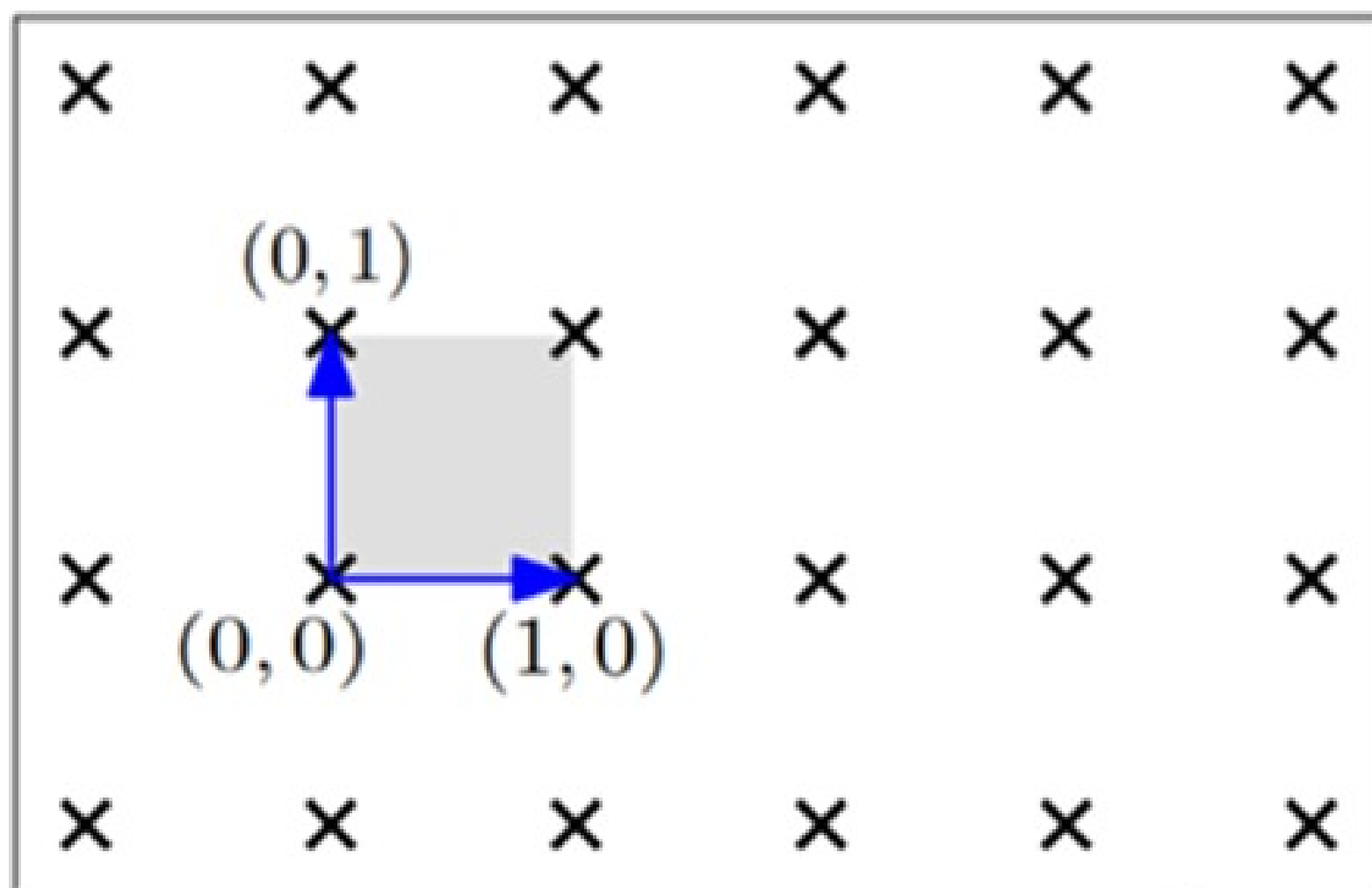
这些元素集合和对应的加法操作 $(L, +)$ 称为格（Lattice）。

这组线性无关向量 B （即 $v_1, v_2, v_3, \dots, v_n$ ）称为格的基，其向量个数被称为格的维度。

1. 格

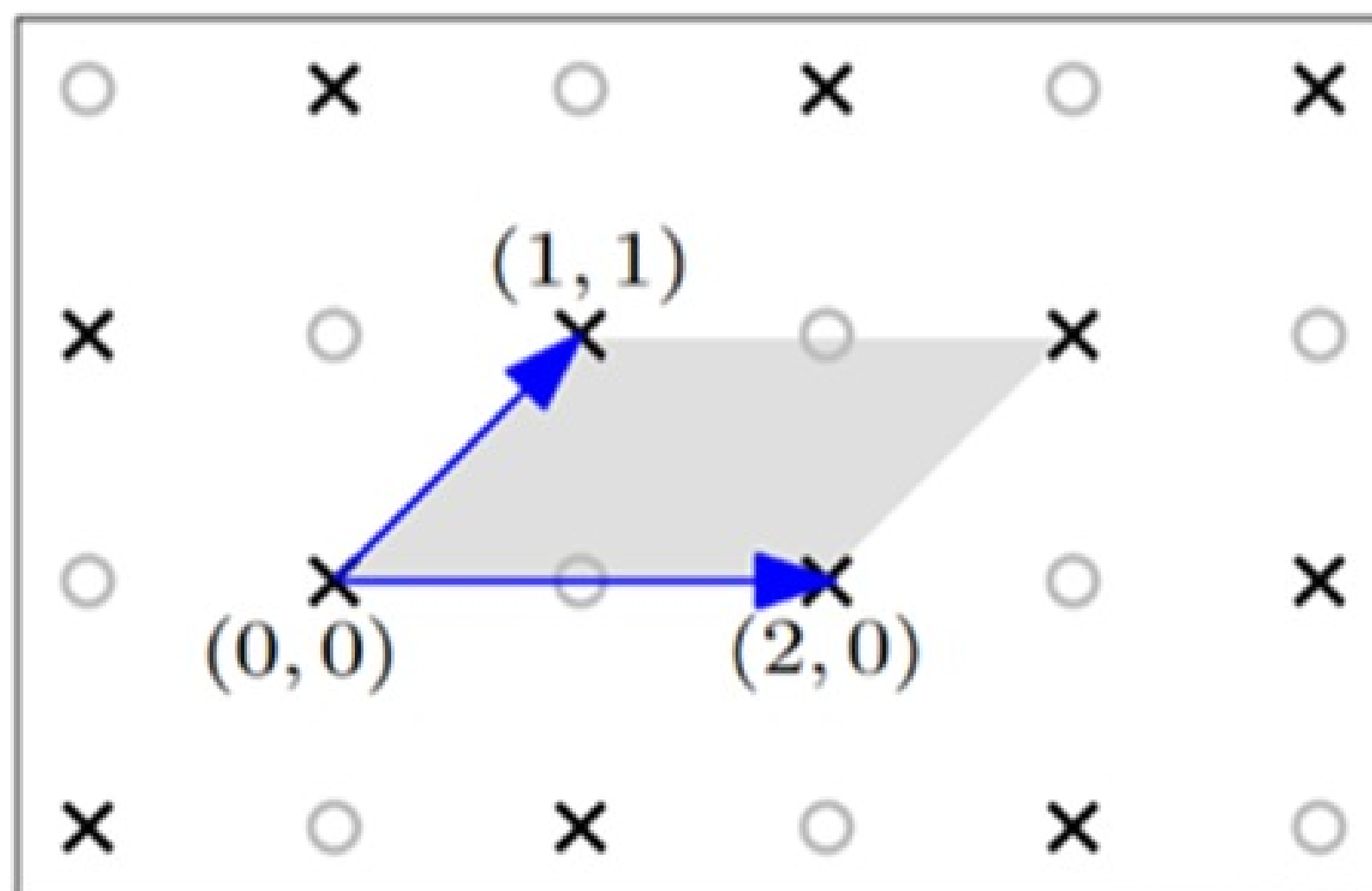
(2) 格的示例

格基 $(1, 0)^T$ 与 $(0, 1)^T$ 可以产生二维空间的所有整数格，如图2.4.1（左）。同时，我们发现，使用格基 $(1, 0)^T$ 与 $(1, 1)^T$ 同样可以生成二维空间的所有整数格，如图2.4.1（右）。也就是说，一个格的基向量可以有多个，图2.4.1（左）的基向量正交程度好一些，称之为“好基”，而图2.4.1（右）的基向量正交程度坏一些，称之为“坏基”。



1. 格

再如，格基 $(1, 1)^T$ 与 $(2, 0)^T$ 不能产生二维空间的所有整数格。如图2.4.2，打“x”的为可产生的格，横纵坐标相加为偶数。



上面都是简单的二维例子，一个格可以由无限多的维度和无限多的向量组成，所以虽然二维看起来非常简单，但是随着基向量和维度的数量增加时，问题很快会变的非常复杂。**一般来说，达到足够安全性的方案，格的维度在1000左右。**

1. 格

(3) 格上的难题

尽管格也由基扩展获得，它和向量空间最大的不同在于，**它的系数限制为整数**，从而生成一系列离散的空间向量。格上的向量的离散性质催生了一系列新的难题。

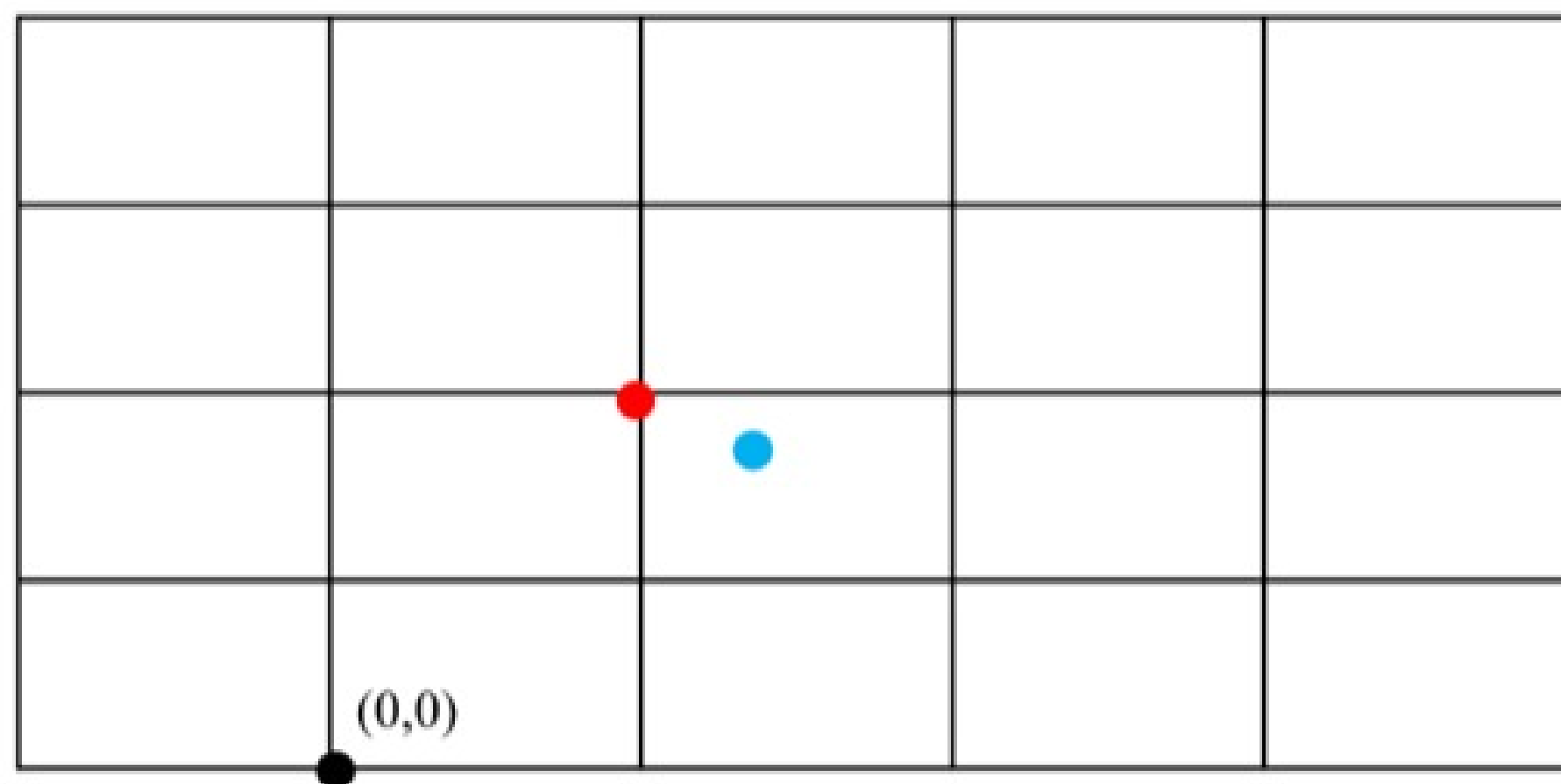
格上的主要难题是最近向量问题（Closest Vector Problem，简称CVP）和最短向量问题（Shortest Vector Problem，简称SVP）。

定义（最近向量问题）： 给定一个格 L 和一个在向量空间 R^n 中但不在格 L 中的向量 $w \in R^n$ ，试图找到一个离 w 最近的向量 $v \in L$ ，即与 w 的欧氏距离最小的向量。

定义（最短向量问题）： 对于给定的格 L ，找到一个非零的格向量 v ，使得对于任意的非零向量 $u \in L$ ，有 $\|v\| \leq \|u\|$ 。

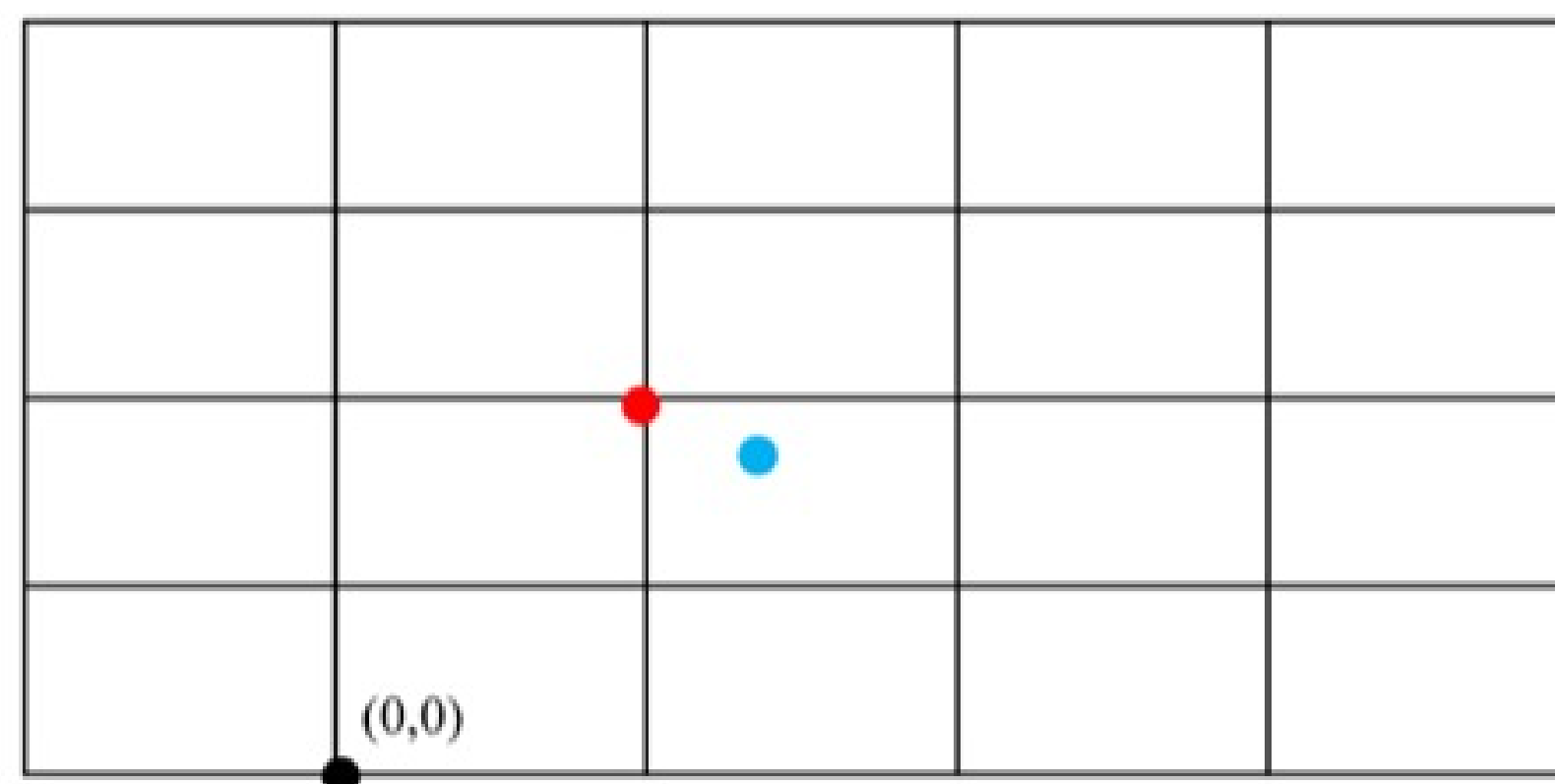
换种说法，该问题试图找到格 L 上一个最短的向量 $v \in L$ ，其与零点的欧氏距离最小。通常使用符号 $\|v\|$ 来表示向量 v 与零点的欧式距离。

1. 格



二维空间里，给定蓝色点表示的非格上的向量，很容易找到红色点表示的格上的向量，距离蓝色点最近。但是，当基向量和维度增加时，寻找最近向量将变的很困难。

1. 格



因此，通过将正交程度差的基作为公钥，将正交程度好的基作为私钥，将解密系统设计为解决格上的最短向量问题或者最近向量问题，就能够提供一种基于格的加密方案。以最近向量问题举例，可以将数据编码到一个安全的格点（比如图2.4.3的红点），加密算法会生成一个离红点近的密文（比如图2.4.3的蓝点），解密的时候拥有私钥的一方可以在多项式时间内找到密文（蓝点）对应的最近向量（红点），但是拥有公钥和其它公开参数的一方在多项式时间内是无法完成求解的。

1. 格

基于格的密码系统具有两个优点：

- 使用线性代数操作实现加解密，具有易实现、高效率的特点。
- 安全性高。为达到 k 个比特的安全等级，传统基于大数分解或离散对数问题的加密系统的加解密操作需要 $O(k^3)$ 的时间复杂度，而基于格的加密系统仅需要 $O(k^2)$ 的时间复杂度。随着量子计算机的问世，大数因式分解之类的经典难题可以在多项式时间被解决，但是量子计算机尚无法在多项式时间内解决格所对应的难题。

2. 理想格

循环格：循环格是一种特殊的格。

循环格最显著的优点就是能够**用一个向量来表示**，可以采用相关算法来加速运算，可以进一步解决基于一般格上的密码方案中密钥量大、运行效率较低的问题。

对于一系列向量 $v_0 = (v_1, v_2, v_3, \dots, v_n)^T, v_1 = (v_n, v_1, v_2, \dots, v_{n-1})^T, v_2 = (v_{n-1}, v_n, v_1, \dots, v_{n-2})^T, \dots$ **，以循环生成的** n **个向量为基生成的格被称为循环格。**

2. 理想格

理想格。理想格是对循环格概念的推广，一般格是群的子群，理想格指该格同时也是环上的理想。在多项式环 $\mathbb{Z}[x]/(f(x))$ 上，循环格的基是通过给出一个多项式 $v \in L$ ，然后对其连续模乘 x 得到 $\{v_i = v_0 * x^i \pmod{f(x)} | i \in [0, n-1]\}$ 。可以证明，在多项式环上构建的循环格即为环的理想，也称为理想格。

理想格有两个优点：

(1) 可以降低格表示的空间尺寸。格的表示方式需要比较大的空间，比如用一个 $n \times n$ 矩阵来表示一组基，则需要存储 n^2 个元素。而理想格的表示则非常简单，对于基而言，给出一个多项式即可。

(2) 理想格是环上的理想，对内具有乘法封闭性，对外有乘法吸收性，这一特点使得理想格很容易构造全同态加密方案。

有关理想格，说法错误的是

- A 使用线性代数操作实现加解密，具有易实现、高效率的特点
- B 理想格是环上的理想，对内具有乘法封闭性，对外有乘法吸收性
- C 理想格的基只有一个
- D 基于理想格构建的密码学方案具有抗量子攻击的特性

3. Gentry方案

- 类同态加密方案可以支持有限次数的各类同态操作，不满足强同态。如果想要不断地进行同态操作，一种简单直接的方法是将该密文解密并且再次加密，从而能够获得一个“全新”的密文，这个过程简称为“刷新”。刷新之后的密文相当于被重置回刚刚加密的状态，从而继续支持更多的同态操作。但是这样的话，需要使用密钥对密文进行解密，这违背了同态加密在加密状态下进行持续运算的原则。
- Gentry博士敏锐地察觉到，如果能够设计一种加密方案，它的**解密操作本身能够做成同态操作**，就能**在全程不解密情况下完成“刷新”操作**，这个操作称为“**自举**”。

2. Gentry方案

Gentry的基于理想格的近似同态加密方案的具体实现如下。

(1) 密钥生成

给定一个多项式环 $R = \mathbb{Z}[x]/(f(x))$ ， R 上的一个理想 I 及其固定基 B_I ，通过循环格生成理想格 J ，满足 $I + J = R$ ，生成两组 J 的基 (B_J^{sk}, B_J^{pk}) 作为公私钥对。其中，私钥 B_J^{sk} 正交化程度较高，公钥 B_J^{pk} 正交化程度较低。另外，提供一个随机函数 $Samp(B_I, x)$ 用于从 $x + B_I$ 的陪集中抽样。最终， $(R, B_I, B_J^{pk}, Samp())$ 为公钥， B_J^{sk} 为私钥。

注： $I + J = R$ 表示和 J 上的元素在运算 $+$ 的情况，结果在 R 上。

2. Gentry方案

(2) 加密：通过函数 $Samp(B_I, x)$ 随机选择向量 r, g ，使用 B_J^{pk} 对明文 $m \in \{0,1\}^n$ 进行加

密，有

$$c = Enc(m) = m + r \cdot B_I + g \cdot B_J^{pk}$$

说明： $g \cdot B_J^{pk}$ 是理想格 J 上的一个元素，因为是基于基 B_J^{pk} 生成的。

(3) 解密：通过私钥 B_J^{sk} 解密密文 c ，得到

$$m = c - B_J^{sk} \cdot \left\lfloor (B_J^{sk})^{-1} \cdot c \right\rfloor (\text{mod } B_I)$$

式中， $\lfloor \cdot \rfloor$ 表示对向量各维度坐标进行四舍五入取整。

2. Gentry方案

2. 正确性

在加密阶段，密文 c 可以看作一个格 J 中的元素 $g \cdot B_J^{pk}$ 加上噪声 $m + r \cdot B_I$ 。

在解密阶段， $B_J^{sk} \cdot \left\lfloor (B_J^{sk})^{-1} \cdot c \right\rfloor$ 为应用**取整估计法**求解**最近向量问题**（CVP），即找到

密文向量**在格 J 中最近的向量**，即 $B_J^{sk} \cdot \left\lfloor (B_J^{sk})^{-1} \cdot c \right\rfloor = g \cdot B_J^{pk}$ ，因此，我们有

$$m = c - B_J^{sk} \cdot \left\lfloor (B_J^{sk})^{-1} \cdot c \right\rfloor \pmod{B_I} = c - g \cdot B_J^{pk} \pmod{B_I} = m + r \cdot B_I \pmod{B_I}$$

注意：应用取整估计法解CVP问题**要求 $m + r \cdot B_I$ 足够小**，才能保证其加密的格元素 $g \cdot$

B_J^{pk} 和解密时找到的最近的格元素是相同元素，即 $B_J^{sk} \cdot \left\lfloor (B_J^{sk})^{-1} \cdot c \right\rfloor = \mathbf{g} \cdot B_J^{pk}$ 。

2. Gentry方案

3. 同态性

在该方案中，明文和密文之间的线性关系使得同态操作易于实现，直接密文相加即可实现同态加法：

$$c_1 + c_2 = m_1 + m_2 + (r_1 + r_2) \cdot B_I + (g_1 + g_2) \cdot B_J^{pk}$$

结果仍在密文空间中，并且只要 $m_1 + m_2 + (r_1 + r_2) \cdot B_I$ 相对较小，即可运用上述解密方法得到明文 $m_1 + m_2$ 。其同态乘法也可以直接使用密文相乘：

$$c_1 \cdot c_2 = e_1 e_2 + (e_1 g_2 + e_2 g_1 + g_1 g_2) \cdot B_J^{pk}$$

其中， $e_1 = m_1 + r_1 \cdot B_I$ ， $e_2 = m_2 + r_2 \cdot B_I$ 。该结果仍然在密文空间中，并且当 $|e_1 \cdot e_2|$ 足够小时，可以通过上述解密方法获得 $m_1 \cdot m_2$ 。

2. Gentry方案

4. 自举

随着加法同态和乘法同态的积累，密文中的噪声项逐渐积累增大，直至无法从密文中解密明文，这个时候就得需要借助自举技术消除噪音，使其支持无限次数的加法和同态乘法。因为Gentry自举技术较为复杂，这里不去详细介绍。

5. 安全性

如上述加解密过程，安全性规约到最近向量问题（CVP）的求解上，只有在格的基正交程度较高的私钥上可以获得尽可能接近的格向量，正交程度较低的基 B_j^{pk} 无法解密明文。

3. CKKS方案

1. 容错学习

- 容错学习 (Learning with Error, LWE) 是在格的难题上构建出来的问题，可以看作解一个带噪声的线性方程组：给定随机向量 $s \in \mathbb{Z}_q^n$ 、随机选择线性系数矩阵 $A \in \mathbb{Z}_q^{n \times n}$ 和随机噪声 $e \in \mathbb{Z}_q^n$ ，生成矩阵线性运算结果 $(A, A \cdot s + e)$ 。
- LWE问题试图从该结果中反推 s 的值，已经证明了LWE至少和格中的难题一样困难，从而能够抵抗量子计算机的攻击。

3. CKKS方案

LWE问题使得在其上构建的加密系统实现十分简单：

密钥生成函数：给定随机向量 $s \in \mathbb{Z}_q^n$ ，随机选择线性系数矩阵 $A \in \mathbb{Z}_q^{n \times n}$ 和随机噪声 $e \in \mathbb{Z}_q^n$ ，将 $(-A \cdot s + e, A)$ 作为公钥， s 作为私钥。

加密函数：对于需要加密的消息 $m \in \mathbb{Z}_q^n$ ，使用公钥加密为

$$(c_0, c_1) = (m - A \cdot s + e, A)。$$

解密函数：使用私钥进行解密，计算 $c_0 + c_1 \cdot s = m + e$ ，当噪声 e 足够小时，能够尽可能地恢复明文 m 。

3. CKKS方案

在LWE基础上，RLWE问题将LWE问题扩展到环结构上。

- RLWE问题在LWE的一维向量空间上，将原来的 $\mathbb{Z}_q$ 环替换成 n 阶多项式环 $\mathbb{Z}_q[X]/(X^N + 1)$ 。于是，LWE问题中的 s, e, m 从 n 个 $\mathbb{Z}_q$ 环中的元素被替换成一个从多项式环中选取的多项式； $n \times n$ 的线性系数矩阵 A 被替换成 1×1 的多项式 $a \in \mathbb{Z}_q[X]/(X^N + 1)$ 。
- 通过转换，RLWE的公钥大小从 $\mathcal{O}(n^2)$ 级下降到 $\mathcal{O}(n)$ 级，而不会减少消息 m 携带的数据量（从 N 维向量替换成一个 n 阶多项式）。另外，基于多项式的乘法操作可以使用离散傅立叶变换算法达到 $\mathcal{O}(n \log(n))$ 的计算复杂度，从而比直接进行矩阵向量乘法快。

3. CKKS方案

2. 方案构造

CKKS层次同态加密方案即是基于上述RLWE问题实现的。具体实现如下：

密钥生成函数： 给定安全参数，CKKS生成私钥 $s \in \mathbb{Z}_q[X]/(X^N + 1)$ 和公钥

$p = (-a \cdot s + e, a)$ 。式中， a, e 皆表示多项式环中随机抽取的元素 $a, e \in$

$\mathbb{Z}_q[X]/(X^N + 1)$ ，且 e 为较小噪声。

3. CKKS方案

加密函数：对于给定的一个消息 $m \in \mathbb{C}^{N/2}$ （表示为复数向量），CKKS需要对其编码，将其映射到多项式环中生成 $r \in \mathbb{Z}[X]/(X^N + 1)$ 。然后，使用公钥对 r 进行加密：

$$(c_0, c_1) = (r - a \cdot s + e, a)$$

解密函数：对密文 (c_0, c_1) ，CKKS使用密钥进行如下解密：

$$\tilde{r} = c_0 + c_1 * s = r + e$$

$\tilde{r}$ 需要经过解码，从多项式环空间反向映射回向量空间 $\mathbb{C}^{N/2}$ 。

当噪声 e 足够小时，可以获得原消息的近似结果。

CKKS支持浮点运算，为保存消息中的浮点数，在编码过程中CKKS设定缩放因子 $\Delta > 0$ ，并将浮点数乘以缩放因子生成整数的多项式项，其浮点值被保存在缩放因子 Δ 中。

3. CKKS方案

CKKS支持同态加法和同态乘法。给定两个密文 ct_1 和 ct_2 ，其对应的同态加法如下：

$$ct_1 + ct_2 = (c_0, c_1) + (c'_0, c'_1) = (c_0 + c'_0, c_1 + c'_1)$$

对应的同态乘法操作如下：

$$ct_1 \cdot ct_2 = (c_0, c_1) \cdot (c'_0, c'_1) = (c_0 \cdot c'_0, c_0 \cdot c'_1 + c_1 \cdot c'_0, c_1 \cdot c'_1)$$

可见，**在进行同态乘法操作后，密文的大小扩增了一半**。因此，每次乘法操作后，

CKKS都需要进行再线性化（Relinearization）和再缩放（Rescaling）操作。

再线性化。再线性化技术能够将扩增的密文 $(c_0 \cdot c'_0, c_0 \cdot c'_1 + c_1 \cdot c'_0, c_1 \cdot c'_1)$ 再次恢复为二元对 (d_0, d_1) ，从而允许进行更多的同态乘法操作。

3. CKKS方案

再缩放。另外，因为在编码消息的过程中使用了缩放因子 Δ ，在进行同态乘法操作时，**两个缩放因子皆为 Δ 的密文相乘，其结果的缩放因子变为 Δ^2 。**如果连续使用同态乘法，缩放因子将会呈指数级上升。**所以，每次乘法操作之后，CKKS都会进行再缩放的操作，将密文值除以 Δ 以将缩放因子从 Δ^2 恢复到 Δ 。**在不断再缩放除以 Δ 的过程中，表示密文值得可用比特每次会下降 $\log(\Delta)$ 比特，直到最终用尽。此时，无法再继续进行同态乘法。

自举。CKKS中的再线性化和再缩放是为了保证缩放因子不变，同时降低噪音，但会造成密文模数减少，所以只能构成有限级全同态方案。CKKS的自举操作能提高密文模数，以支持无限次数的全同态，但是自举成本很高，在满足需求的时候，甚至不需要执行自举操作，后来有一些研究针对CKKS方案的自举操作做了精度和效率的提升。

目 录

『04』

知识点四：类同态BGN方案

『05』

知识点五：全同态典型方案

『06』

知识点六：开发框架SEAL

1. SEAL库安装

(1) git clone 加密库资源

在Ubuntu的home文件夹下建立文件夹seal，进入该文件夹后，打开终端，输入命令：

```
git clone https://github.com/microsoft/SEAL
```

运行完毕，将在seal文件夹下自动建立SEAL这个新文件夹。

(2) 编译和安装

输入命令：

```
cd SEAL
```

```
cmake .
```

```
make
```

```
sudo make install
```

网络原因可能会报错，多尝试几次。该步骤成功后显示如下：

```
-- Installing: /usr/local/include/SEAL-4.0/seal/util/ztools.h  
liuzheli@liuzheli-virtual-machine:~/seal/SEAL$
```

1. SEAL库安装

说明：

- cmake是一种高级编译配置工具，它可以将多个cpp、hpp文件组合构建为一个大工程的语言。它能够输出各种各样的makefile或者project文件，所有操作都是通过编译CMakeLists.txt来完成。
- 通常cmake执行之后，再执行make和make install可以完成开源项目的编译和安装。
- 在make install步骤，相关的头文件和静态库都被安装复制到了/usr/local文件夹下的include和lib文件夹中，以便系统里其它应用程序可以查找到并使用。

2. 简单测试程序

在seal文件夹下建立一个测试文件夹：demo。进而，在demo文件夹下，使用文本编辑器建立一个cpp文件test.cpp，内容如下：

```
#include "seal/seal.h"
#include <iostream>

using namespace std;
using namespace seal;

int main(){

    EncryptionParameters parms(scheme_type::bfv);
    printf("hellow world\n");
    return 0;
}
```


2. 简单测试程序

为了完成test.cpp的编译和执行，需要编写一个CMakeLists.txt文件，内容如下：

```
cmake_minimum_required(VERSION 3.10)
project(demo)
add_executable(test test.cpp)
add_compile_options(-std=c++17)

find_package(SEAL)
target_link_libraries(test SEAL::seal)
```

2. 简单测试程序

编写完毕后，打开控制台，依次运行：

cmake .

make

./test

运行结果如下：

```
liuzheli@liuzheli-virtual-machine: ~/seal/demo
liuzheli@liuzheli-virtual-machine:~/seal/demo$ cmake .
-- Microsoft SEAL -> Version 4.0.0 detected
-- Microsoft SEAL -> Targets available: SEAL::seal
-- Configuring done
-- Generating done
-- Build files have been written to: /home/liuzheli/seal/demo
liuzheli@liuzheli-virtual-machine:~/seal/demo$ make
Consolidate compiler generated dependencies of target test
[100%] Built target test
liuzheli@liuzheli-virtual-machine:~/seal/demo$ ./test
hellow world
liuzheli@liuzheli-virtual-machine:~/seal/demo$
```


3. 应用示例

实验2.3 客户端将一组数据的计算外包到云服务器上，云服务器不能得知密文相关的信息，客户端可以正确完成密文运算结果的解密。

CKKS算法由五个模块组成：密钥生成器keygenerator、加密模块encryptor、解密模块decryptor、密文计算模块evaluator和编码器encoder，其中编码器实现数据和环上元素的相互转换。依据这五个模块，构建同态加密应用的过程为：

- ① 选择CKKS参数parms
- ② 生成CKKS框架context
- ③ 构建CKKS模块keygenerator、encoder、encryptor、evaluator和decryptor
- ④ 使用encoder将数据 n 编码为明文 m
- ⑤ 使用encryptor将明文 m 加密为密文 c
- ⑥ 使用evaluator对密文 c 运算为密文 c'
- ⑦ 使用decryptor将密文 c' 解密为明文 m'
- ⑧ 使用encoder将明文 m' 解码为数据 n

3. 应用示例

```
#include "examples.h"
/*该文件可以在SEAL/native/example目录下找到*/
#include <vector>
using namespace std;
using namespace seal;
#define N 3
//本例目的：给定x, y, z三个数的密文，让服务器计算x*y*z

int main(){
//初始化要计算的原始数据
vector<double> x, y, z;
    x = { 1.0, 2.0, 3.0 };
    y = { 2.0, 3.0, 4.0 };
    z = { 3.0, 4.0, 5.0 };
```

```
/**
客户端的视角：生成参数、构建环境和生成密文
**/

//（1）构建参数容器 parms
EncryptionParameters parms(scheme_type::ckks);
/*CKKS有三个重要参数：
1.poly_module_degree(多项式模数)
2.coeff_modulus（参数模数）
3.scale（规模）*/

size_t poly_modulus_degree = 8192;
parms.set_poly_modulus_degree(poly_modulus_degree);
parms.set_coeff_modulus(CoeffModulus::Create(poly_modulus_degree, { 60, 40, 40, 60 }));
//选用2^40进行编码
double scale = pow(2.0, 40);

//（2）用参数生成CKKS框架context
SEALContext context(parms);
```

3. 应用示例

```
// (3) 构建各模块
// 首先构建keygenerator, 生成公私钥
KeyGenerator keygen(context);
auto secret_key = keygen.secret_key();
PublicKey public_key;
keygen.create_public_key(public_key);

// 构建编码器, 加密模块、运算器和解密模块
// 注意加密需公钥pk; 解密需私钥sk; 编码器需要scale
    Encryptor encryptor(context, public_key);
    Decryptor decryptor(context, secret_key);
    CKKSEncoder encoder(context);
// 对向量x、y、z进行编码
    Plaintext xp, yp, zp;
    encoder.encode(x, scale, xp);
    encoder.encode(y, scale, yp);
    encoder.encode(z, scale, zp);
// 对明文xp、yp、zp进行加密
    Ciphertext xc, yc, zc;
    encryptor.encrypt(xp, xc);
    encryptor.encrypt(yp, yc);
    encryptor.encrypt(zp, zc);
```

```
// 至此, 客户端将pk、CKKS参数发送给服务器, 服务器开始运算
/*****
服务器的视角: 生成重线性密钥、构建环境和执行密文计算
*****/

// 生成重线性密钥和构建环境
SEALContext context_server(parms);
    RelinKeys relin_keys;
    keygen.create_relin_keys(relin_keys);
    Evaluator evaluator(context_server);

/* 对密文进行计算, 要说明的原则是:
- 加法可以连续运算, 但乘法不能连续运算
- 密文乘法后要进行relinearize操作
- 执行乘法后要进行rescaling操作
- 进行运算的密文必需执行过相同次数的
rescaling (位于相同level) */
    Ciphertext temp;
    Ciphertext result_c;
```


3. 应用示例

```
//计算 $x*y$ ，密文相乘，要进行relinearize和rescaling操作
evaluator.multiply(xc,yc,temp);
evaluator.relinearize_inplace(temp, relin_keys);
evaluator.rescale_to_next_inplace(temp);

//在计算 $x*y * z$ 之前，z没有进行过rescaling操作，所以需要
//对z进行一次乘法和rescaling操作，使 $x*y$ 和z在相同层
Plaintext wt;
encoder.encode(1.0, scale, wt);

//执行乘法和rescaling操作：
evaluator.multiply_plain_inplace(zc, wt);
evaluator.rescale_to_next_inplace(zc);

//最后执行 $temp (x*y) * zc (z*1.0)$ 
evaluator.multiply_inplace(temp, zc);
evaluator.relinearize_inplace(temp, relin_keys);
evaluator.rescale_to_next(temp, result_c);
```

```
//计算完毕，服务器把结果发回客户端
/*****
客户端的视角：进行解密和解码
*****/

//客户端进行解密
Plaintext result_p;
decryptor.decrypt(result_c,
result_p);
//注意要解码到一个向量上
vector<double> result;
encoder.decode(result_p,
result);
//得到结果，正确的话将输出：
{6.000, 24.000, 60.000, ..., 0.000,
0.000, 0.000}
cout << "结果是: " << endl;
print_vector(result,3,3);
return 0;
}
```

3. 应用示例

参数解释: (1) **poly_modulus_degree (polynomial modulus)**

该参数必须是2的幂, 如1024, 2048, 4096, 8192, 16384, 32768。更大的参数会增加密文的尺寸, 这会让计算变慢, 但支持的计算也更多。

(2) **[ciphertext] `coefficient modulus**

coeff_modules的个数决定了你能进行rescaling的次数, 进而决定了你能执行的乘法操作的次数。coeff_modules的最大位数与poly_modules有直接关系, 列表如下:

poly_modulus_degree	max coeff_modulus bit-length
1024	27
2048	54
4096	109
8192	218
16384	438
32768	881

3. 应用示例

(2) [ciphertext] `coefficient modulus

本文例子中的{60, 40, 40, 60}有以下含义：

- ① coeff_modules总位长200 (60+40+40+60) 位
- ② 最多进行两次（两层）乘法操作

该系列数字的选择不是随意的，有以下要求：

- ① 总位长不能超过上表限制
- ② 最后一个参数为特殊模数，其值应该与中间模数的最大值相等
- ③ 中间模数与scale尽量相近

注意：如果将模数变大，则可以支持更多层级的乘法运算，比如poly_modulus为16384则可以支持coeff_modules= { 60, 40, 40, 40, 40, 40, 40, 60 }，也就是6层的运算。

(3) Scale

Encoder利用该参数对浮点数进行缩放，每次相乘后密文的scale都会翻倍，因此需要执行rescaling操作约减一部分，约模的大素数位长由coeff_modules中的参数决定。

3. 应用示例

注意事项：

- 每次进行运算前，要保证参与运算的数据位于同一“level”上。加法不需要进行 rescaling 操作，因此不会改变数据的 level。数据的 level 只能降低无法升高，所以要**小心设计计算的先后顺序。**
- SEAL 提供了 reverse、square 等有限的计算操作，大部分复杂运算需要自己编写代码实现，在实现过程中要根据数据量把握好精度和性能的取舍。

谢 谢

